Assignment 13.4

Name: Sunny Kolluri

HTNO: 2303A52444

Batch: 43

Task 1:

Prompt: I have a legacy Python code that uses a loop to double values in a list.

values = [2, 4, 6, 8, 10]

doubled = []

for v in values:

  doubled. Append (v * 2)

print(doubled)

Refactor this code using a more Pythonic approach such as list comprehensions while keeping the same output. Also explain the improvement.

```python
# I have a legacy Python code that uses a loop to double values in a list.
#values = [2, 4, 6, 8, 10]
#doubled = []
#for v in values:
#    doubled.append(v * 2)
#print(doubled)
# Refactor this code using a more Pythonic approach such as list comprehensions while keeping the same output. Also explain the improvement
values = [2, 4, 6, 8, 10]
doubled = [v * 2 for v in values]
print(doubled)
# Explanation:
# The refactored code uses a list comprehension, which is a more concise and readable way to create a new list by applying an expression to each ite
# m in an iterable.
# This approach eliminates the need for an explicit loop and the append method, making the code cleaner and easier to understand.
# Additionally, list comprehensions are often faster than traditional loops because they are optimized in Python's implementation.
```

```
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/python.exe" "c:/Users/navya/OneDrive/Desktop/AI
 Assit/Assignment 13.4.py"
[4, 8, 12, 16, 20]
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation:

The legacy code used a loop and the append() method to double each value in the list.
After refactoring, a list comprehension was used to perform the same operation in a single line.
This makes the code shorter, more readable, and more Pythonic while producing the same output.

Task 2:

Prompt: I have a Python script that builds a sentence using repeated string concatenation inside a loop.

words = ["Refactoring", "with", "AI", "improves", "quality"]

message = ""

for w in words:

  message += w + " "

print (message. Strip ())

Suggest a more efficient and readable way to write this code while keeping the output the same. Explain the improvement.

```python
#I have a Python script that builds a sentence using repeated string concatenation inside a loop.
#words = ["Refactoring", "with", "AI", "improves", "quality"]
#message = ""
#for w in words:
#    message += w + " "
#print (message. Strip ())
#Suggest a more efficient and readable way to write this code while keeping the output the same. Explain the improvement.
words = ["Refactoring", "with", "AI", "improves", "quality"]
message = " ".join(words)
print(message)
# Explanation:
# The refactored code uses the join method, which is a more efficient way to concatenate strings in Python.
# This method is optimized for string concatenation and avoids the overhead of creating multiple
# intermediate string objects that occur with repeated concatenation in a loop.
# Using join is also more readable as it clearly indicates that we are combining a list of words into a
#  single string with spaces in between, improving the clarity of the code.
```

```
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.
 Assit/Assignment 13.4.py"
Refactoring with AI improves quality
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation:

The original code used repeated string concatenation inside a loop, which is inefficient.
The refactored code uses the join () method to combine all words into a single string.
This approach is faster, cleaner, and easier to understand.

Task 3:

Prompt:

I have a Python configuration dictionary where some keys may not exist.

config = {"host": "localhost", "port": 8080}

if "timeout" in config:

print(config["timeout"])

else:

print ("Default timeout used")

Refactor this code using safer dictionary access methods such as dict. Get () while maintaining the same behaviour. Explain why it is better.

```
#I have a Python configuration dictionary where some keys may not exist.
#config = {"host": "localhost", "port": 8080}
#if "timeout" in config:
 #   print(config["timeout"])
#else:
#    print ("Default timeout used")
#Refactor this code using safer dictionary access methods such as dict. Get () while maintaining the same behaviour. Explain why it is better.
config = {"host": "localhost", "port": 8080}
timeout = config.get("timeout", "Default timeout used")
print(timeout)
# Explanation:
# The refactored code uses the get method of the dictionary, which allows for safer access to keys that may not exist.
# The get method takes a second argument that specifies a default value to return if the key is not found,
# eliminating the need for an explicit check using an if statement.
# This approach is more concise and reduces the chances of errors,
# such as KeyError, that can occur when trying to access a non-existent key directly.
```

```
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/
 Assit/Assignment 13.4.py"
Default timeout used
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation:

The legacy code checked if a key exists in the dictionary using an if condition before accessing it.
The refactored code uses dict. Get (), which safely returns a default value if the key is missing.
This makes the code shorter, safer, and reduces the chance of errors like Keyerror.

Task 4:

Prompt: I have Python code with multiple if-elif conditions to perform operations.

```
action = "divide"

x, y = 10, 2

if action == "add":

    result = x + y

elif action == "subtract":

    result = x - y

elif action == "multiply":

    result = x * y

elif action == "divide":

    result = x / y

else:

    result = None

print(result)
```

Suggest a cleaner and more scalable approach using mapping techniques such as dictionaries of functions. Keep the same output and explain the improvement.

```python
def add(x, y):
    return x + y
def subtract(x, y):
    return x - y
def multiply(x, y):
    return x * y
def divide(x, y):
    return x / y
action = "divide"
x, y = 10, 2
operations = {
    "add": add,
    "subtract": subtract,
    "multiply": multiply,
    "divide": divide
}
result = operations.get(action, lambda x, y: None)(x, y)
print(result)
# Explanation:
# The refactored code uses a dictionary to map action strings to their corresponding functions, which
# allows for a cleaner and more scalable approach to handling multiple conditions.
# This method eliminates the need for multiple if-elif statements and makes it easier to add new
# operations in the future by simply adding new entries to the dictionary.
# Using a dictionary of functions also improves readability by clearly associating actions with their operations,
# it reduces the likelihood of errors that can arise from complex conditional logic.
```

```
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/
 Assit/Assignment 13.4.py"
5.0
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation:

The original code used multiple if–elif statements to perform different operations.
The refactored version uses a dictionary mapping actions to functions, making the code more organized.
This approach is more scalable and easier to extend if new operations need to be added.

Task 5:

Prompt: I have Python code that searches for an item in a list using a manual loop.

inventory = ["pen", "notebook", "eraser", "marker"]

found = False

for item in inventory:

```
    if item == "eraser":
        found = True
        break
print ("Item Available" if found else "Item Not Available")
```

Refactor this code into a more concise and readable Pythonic approach while keeping the same behaviour and output. Explain the improvement.

```python
inventory = ["pen", "notebook", "eraser", "marker"]
item_to_find = "eraser"
print("Item Available" if item_to_find in inventory else "Item Not Available")
# Explanation:
# The refactored code uses the in keyword, which is a more concise and readable way to check for the presence of an item in a list.
# This approach eliminates the need for a manual loop and a separate variable to track whether the item
# was found, making the code cleaner and easier to understand.
# Using the in keyword is also more efficient as it is optimized for membership testing in Python,
# and it reduces the chances of errors that can occur with manual loops,
# such as forgetting to break out of the loop or incorrectly updating the found variable.
```

```
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.v
 Assit/Assignment 13.4.py"
Item Available
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation:

The legacy code used a manual loop and a flag variable to check if an item exists in a list.
The refactored code uses the in keyword to directly check membership in the list.
This method is simpler, cleaner, and more Pythonic.